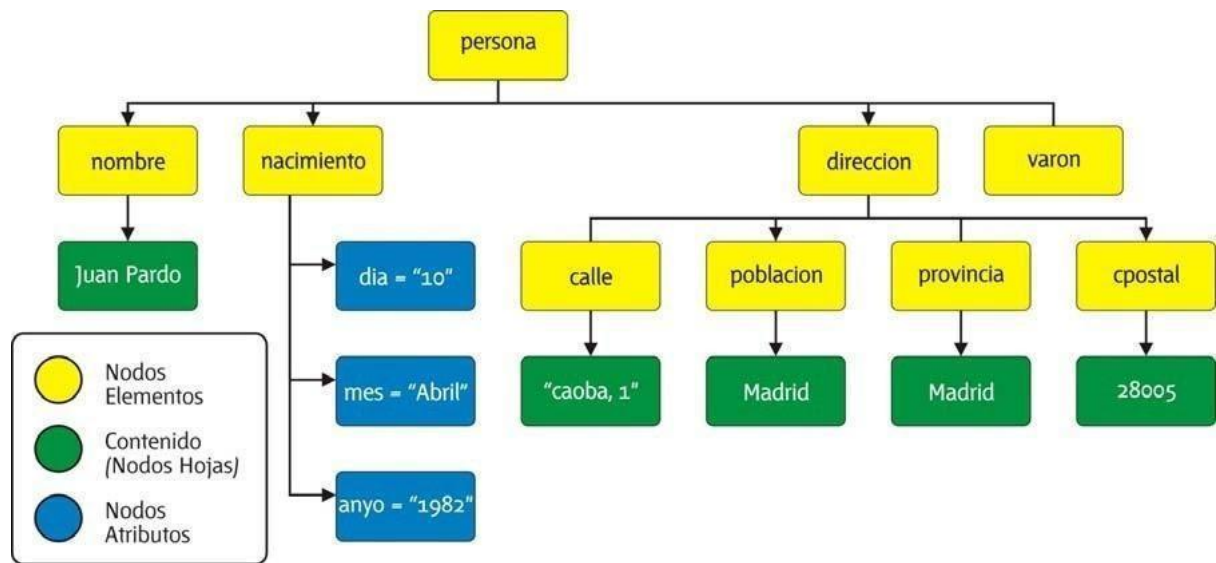


Ejercicio 1

Hacer una JSON que pueda contener una lista de personas, cada persona está formada por los siguientes elementos. **Hacer un ejemplo con 2 personas**



Conversión JSON:

```
{
  "personas": [
    {
      "nombre": "María",
      "apellido": "Majano",
      "edad": 26,
      "email": "maria.majano@gmail.com"
    },
    {
      "nombre": "Pedro",
      "apellido": "Ruíz",
      "edad": 28,
      "email": "pedro.ruiz@gamil.com"
    }
  ]
}
```

It aims to optimize the programming process and enhance development efficiency through AI. [LEARN MORE >](#)

To format and validate your JSON, just copy + paste it below:

```
1 {
2   "personas": [
3     {
4       "nombre": "Maria",
5       "apellido": "Majano",
6       "edad": 26,
7       "email": "maria.majano@gmail.com"
8     },
9     {
10      "nombre": "Pedro",
11      "apellido": "Ruiz",
12      "edad": 28,
13      "email": "pedro.ruiz@gamil.com"
14    }
15  ]
16 }
```

[Validate JSON](#) [Clear](#) [Compress](#)

JSON is valid!

Ejercicio 2

Una aplicación de mensajería quiere contener cada mensaje en un JSON. Cada mensaje tiene estos campos



Ha un JSON con un mensaje de ejemplo

Conversión JSON:

```
{
  "mensaje": {
    "remitente": {
      "nombre": "Inés Pérez",
      "email": "ines.perez@email.com"
    },
    "destinatario": {
      "nombre": "Lucas Martín",
      "email": "lucas.martin@email.com"
    },
    "asunto": "Reunión",
    "texto": "Hola Lucas, te escribo para confirmar la reunión de seguimiento programada para mañana a las 10:00 a.m. en la sala de conferencias. ¡Nos vemos pronto!"
  }
}
```

The screenshot shows a web browser with three tabs: 'Página de inicio | EducamosCLU', 'JSON Online Validator and Form...', and 'Ejercicios JSON - Documentos'. The address bar shows 'jsonlint.com'. The page has a header with the Trae logo and the text 'From Prompt to Production' with a 'LEARN MORE >' button. Below the header, it says 'To format and validate your JSON, just copy + paste it below:'. A text area contains the JSON code from the previous block, with line numbers 1 through 14 on the left. A scrollbar is visible at the bottom of the text area. Below the text area are three buttons: 'Validate JSON' (green), 'Clear' (light blue), and 'Compress' (light blue). At the bottom, a green banner displays the message 'JSON is valid!'.

```
1 {
2   "mensaje": {
3     "remitente": {
4       "nombre": "Inés Pérez",
5       "email": "ines.perez@email.com"
6     },
7     "destinatario": {
8       "nombre": "Lucas Martín",
9       "email": "lucas.martin@email.com"
10    },
11    "asunto": "Reunión",
12    "texto": "Hola Lucas, te escribo para confirmar la reunión de seguimiento programada para mañana a la
13  }
14 }
```

Validate JSON Clear Compress

JSON is valid!

Ejercicio 3 (Pedidos)

Genera el JSON necesario para una app de compras que debe enviar la información de **cada pedido** al servidor en un archivo JSON. Para cada pedido

se debe generar un archivo JSON que contendrá:

- Un número de pedido, la fecha (AAAA-MM-DD) y la hora del pedido (hh:mm:ss)
- Un elemento usuario que contendrá, el nombre de usuario, el nombre completo y la dirección de envío.
- La dirección de envío a su vez es un elemento que está compuesto por la calle, la población y el código postal.

- A continuación, irá una lista de artículos. Para cada artículo se incluirá el código de artículo, una descripción y el precio.

Haz un ejemplo de archivo para un pedido de tres artículo

Conversión JSON:

```
{
  "numero_pedido": "12345",
  "fecha": "2025-04-14",
  "hora": "13:45:00",
  "usuario": {
    "nombre_usuario": "ana_95",
    "nombre_completo": "Ana Valero",
    "direccion_envio": {
      "calle": "Calle Arenal 12",
      "poblacion": "Madrid",
      "codigo_postal": "28013"
    }
  },
  "articulos": [
    {
      "codigo": "A54545",
      "descripcion": "Falda rayas",
      "precio": 15.99
    },
    {
      "codigo": "A4845",
      "descripcion": "Pantalón vaquero",
      "precio": 25.95
    },
    {
      "codigo": "A6548",
      "descripcion": "Sandalias tacón",
      "precio": 35.99
    }
  ]
}
```

Página de inicio | EducamosCL

JSON Online Validator and For

Ejercicios JSON - Documentos

jsonlint.com

AI Coding, Elevated

LEARN MORE >

To format and validate your JSON, just copy + paste it below:

```
1 {
2   "numero_pedido": "12345",
3   "fecha": "2025-04-14",
4   "hora": "13:45:00",
5   "usuario": {
6     "nombre_usuario": "ana_95",
7     "nombre_completo": "Ana Valero",
8     "direccion_envio": {
9       "calle": "Calle Arenal 12",
10      "poblacion": "Madrid",
11      "codigo_postal": "28013"
12    }
13  },
14  "articulos": [
15    {
16      "codigo": "A54545",
17      "descripcion": "Falda rayas",
18      "precio": 15.99
19    },
20    {
21      "codigo": "A4845",
22      "descripcion": "Pantalón vaquero",
23      "precio": 25.95
24    },
25    {
26      "codigo": "A6548",
27      "descripcion": "Sandalias tacón",
28      "precio": 35.99
29    }
30  ]
31 }
```

Validate JSON

Clear

Compress

JSON is valid!

Ejercicio 4 (Concesionarios)

Se desea modelar en JSON la información relativa a distintos concesionarios de vehículos de una marca. La lista de concesionarios debe tener al menos uno, no puede estar vacía. Deberá poder recoger la información de contacto de cada concesionario (nombre, dirección, localidad y uno o más teléfonos) así como la lista de los vehículos que tiene a la venta. La lista de vehículos puede estar vacía, o contener 1 o varios vehículos diferentes. Cada vehículo se identifica

mediante su matrícula y pueden ser del tipo “turismo” o “furgoneta”, siendo el

valor por defecto “turismo”. Del vehículo se recoge el modelo, cilindrada, tipo de combustible (sólo puede ser Gasolina

o Diesel), color, y puede contener un elemento “asegurado” que es un booleano

Hacer un ejemplo con 2 concesionarios con 2 y 3 vehículos respectivamente

Conversión JSON:

```
{
  "concesionarios": [
    {
      "nombre": "AutoToledo",
      "direccion": "Av. De la Mancha 12",
      "localidad": "Toledo",
      "telefonos": ["925658779"],
      "vehiculos": [
        {
          "matricula": "1234ABC",
          "modelo": "Seat Ibiza",
          "cilindrada": 1200,
          "combustible": "Gasolina",
          "color": "Negro",
          "asegurado": true
        },
        {
          "matricula": "5678KWP",
          "tipo": "furgoneta",
          "modelo": "Renault Kangoo",
          "cilindrada": 1600,
          "combustible": "Diesel",
          "color": "Blanco"
        }
      ]
    },
    {

```

```
"nombre": "MotorMadrid",
"direccion": "Calle Norte 2",
"localidad": "Madrid",
"telefonos": ["933566987", "925884652"],
"vehiculos": [
  {
    "matricula": "2548JUG",
    "modelo": "Ford Fiesta",
    "cilindrada": 1400,
    "combustible": "Gasolina",
    "color": "Negro",
    "asegurado": false
  },
  {
    "matricula": "2554PRF",
    "tipo": "furgoneta",
    "modelo": "Peugeot Partner",
    "cilindrada": 1700,
    "combustible": "Diesel",
    "color": "Gris"
  },
  {
    "matricula": "5523KÑP",
    "modelo": "Volkswagen Polo",
    "cilindrada": 1300,
    "combustible": "Gasolina",
    "color": "Azul"
  }
]
}
```

It aims to optimize the programming process and enhance development efficiency through AI. [LEARN MORE >](#)

To format and validate your JSON, just copy + paste it below:

```
1 {
2   "concesionarios": [
3     {
4       "nombre": "AutoToledo",
5       "direccion": "Av. De la Mancha 12",
6       "localidad": "Toledo",
7       "telefonos": [
8         "925658779"
9       ],
10      "vehiculos": [
11        {
12          "matricula": "1234ABC",
13          "modelo": "Seat Ibiza",
14          "cilindrada": 1200,
15          "combustible": "Gasolina",
16          "color": "Negro",
17          "asegurado": true
18        },
19        {
20          "matricula": "5678KWP",
21          "tipo": "furgoneta",
22          "modelo": "Renault Kangoo",
23          "cilindrada": 1600,
24          "combustible": "Diesel",
25          "color": "Blanco"
26        }
27      ]
28    }
29  ]
30 }
```

[Validate JSON](#) [Clear](#) [Compress](#)

JSON is valid!

Ejemplo 5 (Videoclub)

Una cadena de videoclubs quiere emplear una base de datos para almacenar información referente a las facturas que se hacen a los clientes. Esta información es la siguiente:

- En un mismo documento se puede guardar información de varias facturas.
- Cada factura está formada por dos tipos de información: datos de cliente y datos del ticket de factura propiamente dichos.
- De los datos del cliente se desea guardar: su nombre, su primer y segundo apellidos, DNI y teléfono (uno o varios). Además, como características del cliente, se desea conocer el identificador de cliente.
- En cada factura habrá alquileres, compras o los dos. Incluye la forma de pago y el importe total.
- Los alquileres se realizan de películas. El alquiler de películas lleva asociada una fecha de devolución que es común a todas las películas alquiladas en la misma factura.

- De cada película se quiere conocer su título, género, duración y los nombres y apellidos de dos actores que participan en ella. Existen dos atributos que definen a las películas: idPelícula y valoración.
- Además, opcionalmente se puede guardar para cada película el nombre de un archivo con la imagen de la carátula en formato jpg.
- Con respecto a las compras, puedes ser de DVDs o cintas de video pero no los dos a la vez.
- De los DVDs interesa el título del DVD, la fecha de salida al mercado y si viene o no con extras (aunque es opcional)
- De las cintas de video se guardará el título, el formato (VHS, por ejemplo) y si está rebobinada o no.

Hacer un ejemplo con dos facturas, una para un cliente por el alquiler de dos películas y otro de la compra de un DVD

Conversión JSON:

```
{
  "facturas": [
    {
      "cliente": {
        "id": "C001",
        "nombre": "Manuela",
        "apellidos": ["Perea", "Marín"],
        "dni": "12345678A",
        "telefonos": ["600123456", "911223344"]
      },
      "alquileres": {
        "fecha_devolucion": "2025-04-22",
        "peliculas": [
          {
            "idPelícula": "P001",
            "titulo": "Harry Potter y la piedra filosofal",
            "genero": "Fantasía",
            "duracion": 152,
            "actores": [
```

```
{
  "nombre": "Daniel", "apellido": "Radcliffe"},
  {"nombre": "Emma", "apellido": "Watson"}
],
"valoracion": 4.8,
"caratula": "harrypotter_piedrafilosofal.jpg"
},
{
  "idPelicula": "P002",
  "titulo": "Troya",
  "genero": "Bélico",
  "duracion": 163,
  "actores": [
    {"nombre": "Brad", "apellido": "Pitt"},
    {"nombre": "Eric", "apellido": "Bana"}
  ],
  "valoracion": 4.9
}
]
},
"forma_pago": "Tarjeta",
"importe_total": 5.50
},
{
  "cliente": {
    "id": "C002",
    "nombre": "Javier",
    "apellidos": ["García", "Rodríguez"],
    "dni": "87654321B",
    "telefonos": ["622334455"]
  },
}
```

```

"compras": {
  "tipo": "DVD",
  "titulo": "Malditos bastardos",
  "fecha_salida": "2009-07-28",
  "extras": true
},
"forma_pago": "Efectivo",
"importe_total": 14.95
}
]
}

```

Página de inicio | EducamosCL | JSON Online Validator and Formater | Ejercicios JSON - Documentos

jsonlint.com

Trae From Prompt to Production [LEARN MORE >](#)

To format and validate your JSON, just copy + paste it below:

```

1 {
2   "facturas": [
3     {
4       "cliente": {
5         "id": "C001",
6         "nombre": "Manuela",
7         "apellidos": [
8           "Perea",
9           "Marín"
10        ],
11        "dni": "12345678A",
12        "telefonos": [
13          "600123456",
14          "911223344"
15        ]
16      },
17      "alquileres": {
18        "fecha_devolucion": "2025-04-22",
19        "peliculas": [
20          {
21            "idPelicula": "P001",
22            "titulo": "Harry Potter y la piedra filosofal",
23            "genero": "Fantasía",
24            "duracion": 152,
25            "actores": [
26              {
27                "nombre": "Daniel",
28                "apellido": "Radcliffe"

```

[Validate JSON](#)
[Clear](#)
[Compress](#)

JSON is valid!

